SIGNVISION: A REAL-TIME COMPUTER VISION AND CONVERSATIONAL AI

FRAMEWORK FOR CONTINUOUS INDIAN SIGN LANGUAGE COMMUNICATION

DEEPAK A – 24619210
MUKESH P – 24619219
DURGA G – 24619211

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"GLASSBOX: PERSONALIZED CALIBRATION FOR PRIVACY-PRESERVING AND EXPLAINABLE RISK SCORING IN ONLINE EXAM PROCTORING"** is the bonafide work of **SHAMA BEGUM S – 24619251, J. S. DURGESH – 24619212 and T. ARUTKUMARAN – 24619204** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., (Ph.D)

Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR
Ms. Mohana Priya P, M.E, (Ph.D)

Assistant Professor
Department of AI&ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on…… at Vels Institute of Science, Technology and Advanced Studies (VISTAS) ,Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and a proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, and we thank our FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, with great pleasure for it. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya,** for the continuous support given to this project, from initial advice and contacts in the early stages of conceptual inception, and through ongoing advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE**, **Ms. P. Mohanapriya Ass. Prof/AI&ML**, who has patiently guided us and provided invaluable advice and help throughout the design and development of this project. We thank the Department of Computer Science and Engineering, VISTAS, for the resources made available for this work. would like to express our greatest gratitude to our staff members, the Librarian, and the non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not least, we wish to thank our parents and friends for their individual support and interest, who inspired and encouraged us to go our own way; without them we would be unable to complete this project.

DEEPAK A – 24619210
MUKESH P – 24619219
DURGA G – 24619211

TABLE OF CONTENTS

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF TABLES	ii
	ABBREVIATIONS	iii
1	**INTRODUCTION**	1
	1.1 Overview of the project	1
	1.2 Scope and objective	3
2	**LITERATURE REVIEW**	6
	2.1 Literature Overview	6
	2.2 Literature Survey	6
3	**SYSTEM DESIGN**	9
	3.1 Existing System	9
	3.2 Proposed System	9
	3.3 System Architecture	10
	3.4 Hardware Requirements	13
	3.5 Software Requirements	14
4	**IMPLEMENTATION**	16

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	4.1 Methodology	16
	4.2 Implementation	19
	4.2.1 Hand Detection Module	19
	4.2.2 Overlap Resolution Module	20
	4.2.3 Continuous Recognition Module	20
	4.2.4 Caption Generation Module	20
	4.2.5 Chatbot Module	21
	4.2.6 Voice Assistant and Avatar Modules	21
	4.2.7 Teacher, Dataset and Training Modules	21
	4.2.8 Desktop and Web Front Ends	21
5	**CONCLUSION AND FUTURE WORK**	22
	5.1 Conclusion	22
	5.2 Future Work	23
	5.3 Source Code	24
	5.4 Results	25
	REFERENCES	28

ABSTRACT

Communication between Indian Sign Language (ISL) users and people who do not know ISL still depends largely on human interpreters, who are too few to be present at every classroom, hospital counter or government office where they are needed. Camera-based sign recognition has been proposed as an alternative, but most existing systems classify isolated alphabets or single-word gestures and stop there – they do not carry a conversation. SignVision addresses this: MediaPipe-based dual-hand landmark detection feeds a temporal buffer and a pluggable sequence classifier built around a two-layer LSTM. Recognized words are grammar-corrected into captions and handed to a locally hosted large language model (via Ollama) that replies with context, closing the loop with synchronized speech and on-screen text. The system runs identically from an OpenCV desktop window or a Flask web interface, and every component – vision, language and speech – executes on-device with no cloud dependency. We describe the architecture, the processing pipeline, and the current implementation status: the perception and language layers are complete and verified with live and injected captions, while the ISL sequence classifier itself awaits a trained model once dataset collection is finished. We outline the evaluation protocol that will be used once training data is available and discuss the honest limitations of a single-user, locally hosted prototype.

Keywords: Indian Sign Language, Hand Landmark Detection, Continuous Sign Recognition, LSTM, MediaPipe, Conversational AI.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	SignVision layered system architecture	11
Figure 3.2	End-to-end processing pipeline of SignVision	12
Figure 3.3	Per-frame feature vector and 30-frame sliding window	13
Figure 4.1	Prediction gating and sentence segmentation logic	17
Figure 4.2	Caption-to-chatbot flow in the web front end	18
Figure 5.1	Working web prototype: live dual-hand tracking, caption and chat panel	25
Figure 5.2	Web interface: live dual-hand landmark overlay with caption panel	26
Figure 5.3	Web interface: local-LLM chat assistant panel	26

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Literature survey and comparison with SignVision	6
Table 2.2	Summary of the base paper	7
Table 3.1	Hardware requirements	13
Table 3.2	Software requirements	14
Table 3.3	Software modules	14
Table 4.1	Key pipeline parameters	17
Table 4.2	Web front-end endpoints	18
Table 5.1	Module-wise implementation status	25
Table 5.2	Comparison of base paper with proposed SignVision	26
Table 5.3	Recognition performance metrics	27

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BGR / RGB	Blue-Green-Red / Red-Green-Blue colour order
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
EMA	Exponential Moving Average
FPS	Frames Per Second
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
ISL	Indian Sign Language
JSON	JavaScript Object Notation
LLM	Large Language Model
LSTM	Long Short-Term Memory
MJPEG	Motion JPEG
NLP	Natural Language Processing
OpenCV	OpenSource Computer Vision Library
REST	Representational State Transfer
TTS	Text-to-Speech
UI	User Interface
WH- words	Question words such as what, where, who, why

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

Indian Sign Language (ISL) is the primary language of India's Deaf and hard-of-hearing community. It is a full visual-gestural language with its own grammar and word order – not a signed version of English or Hindi – and it is used by a large signing population across the country. Outside that community, however, very few people understand ISL, and the result is a communication gap that shows up in ordinary places: a classroom where a teacher cannot follow a student's question, a hospital counter where a patient cannot describe a symptom, a government office where a form cannot be explained.Human interpreters exist, but they are scarce relative to demand and cannot be present at every counter or every classroom. Camera-based sign recognition has been proposed for years as a partial substitute, and recent advances in real-time hand-landmark tracking – MediaPipe in particular – have made it practical to run this kind of recognition on an ordinary laptop webcam without any specialized depth sensor or glove.

This project, **SignVision**, is an AI-based hand sign translation system. It performs continuous ISL recognition from a live webcam feed, corrects the raw output into a readable caption, sends the finalized caption to a locally hosted large language model for a context-aware reply, and delivers that exchange back to the user as synchronized speech, on-screen captioning and a placeholder signing avatar. The full loop – camera to hand landmarks to continuous recognition to caption to conversational reply to voice – runs identically from a desktop OpenCV window and from a browser, and every stage of it, including the language model, runs locally, with no cloud API and no per-request cost.

Problem Statement

Real-time interaction between ISL signers and non-signers is difficult for several concrete reasons:

- **Scarcity of interpreters:** human interpreters are too few to be present at every classroom, hospital counter or government office.
- **Isolated-gesture focus:** most existing recognition software supports isolated alphabets or single-word gestures rather than continuous signing.
- **Small vocabularies:** the recognizable vocabulary of such systems is usually small.
- **Sensitivity to conditions:** recognition is sensitive to lighting and to one hand briefly occluding the other; signer style, hand size and signing speed vary considerably between individuals.
- **One-way interaction:** even where recognition works, there is little to no contextual understanding of what was said, so the interaction stays one-way.

Segmenting a continuous stream of hand motion into distinct signs is significantly harder than classifying one isolated gesture. ISL, like any language, is signed continuously, with one gesture flowing into the next, and the boundaries between words are not marked by any obvious pause. A second gap is just as important: a system that recognizes signs perfectly still only produces a transcript, not a response. Pairing recognition with a conversational layer turns the system from a one-way dictionary lookup into something closer to an actual conversation partner, and this second half is what most existing ISL projects leave out entirely.

Objectives of the Project

The main objectives of this project are to:

- Develop a real-time, camera-based ISL recognition module that runs on ordinary laptop hardware.
- Support sequential, continuous sign interpretation rather than isolated-gesture classification.
- Convert recognized signs into a grammatically corrected, readable text caption.

- Integrate a conversational layer that replies to a finalized caption with contextual relevance.
- Provide synchronized text and speech output to the user.
- Establish an evaluation protocol suited to measuring the system once a trained classifier is available.

Contributions of the Work

- A fixed-length, 126-value per-frame feature representation (two hands × 21 landmarks × 3 coordinates) that lets a 30-frame sliding window be fed to a sequence model regardless of how many hands are visible in a given frame.
- A pluggable sequence-classifier interface that separates the engineering of the recognition pipeline (buffering, striding, confidence gating, repeat suppression, pause-based sentence segmentation) from the ISL classification model itself.
- A rule-based caption grammar corrector that produces a readable sentence from a raw word stream without depending on an external NLP library.
- Integration of a locally hosted conversational LLM, served through Ollama, triggered only on a finalized caption, with a graceful fallback rather than a silent failure.
- Two functionally identical front ends – an OpenCV desktop window and a Flask web page with an MJPEG video stream – built on one shared pipeline.

System Overview

SignVision is organized into three layers plus a local data store, all running on a single machine:

- **Sign Recognition Module:** hand detection (MediaPipe), landmark extraction and preprocessing, and sign classification (LSTM).
- **Text and Language Module:** caption generation, sentence assembly, the chatbot engine (a locally hosted Ollama LLM) and context-aware response generation.
- **Output Module:** text display on the web interface and speech output through an offline text-to-speech engine.

These modules communicate directly, with no network hop between them, because everything executes in one Python process. The only network traffic is the local request to the Ollama server and, for the web front end, the HTTP traffic between the browser and the local Flask server.

Advantages of the System

- **Continuous recognition:** designed for sentence-level signing rather than isolated gestures.
- **Two-way communication:** the conversational layer produces a context-aware reply instead of a bare transcript.
- **Fully on-device:** vision, language and speech all execute locally with no cloud dependency and no per-request cost.
- **No special hardware:** works with an ordinary laptop webcam without a depth sensor, glove or discrete GPU.
- **Graceful degradation:** every optional component has a fallback, so a missing component never crashes the rest of the pipeline.
- **Swappable model:** the classifier sits behind an interface, so a trained model can be plugged in without touching any other module.

Applications of the Project

- **Classrooms:** allowing a teacher to follow a signing student's question.
- **Hospital counters:** allowing a patient to describe a symptom to staff who do not know ISL.
- **Government offices and public services:** allowing a signer to have a form or procedure explained.

- **Learning support:** the teacher module scores practice attempts so that learners can practice signs.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of this project is to design, implement and evaluate a real-time communication assistant that bridges Indian Sign Language users and the wider community. The study aims to move beyond isolated-gesture classification towards continuous, conversational interaction by:

- Extracting hand landmarks from a live webcam feed and turning them into a fixed-length temporal feature representation.
- Segmenting a continuous stream of signs into words and sentences using confidence gating, repeat suppression and pause detection.
- Converting the recognized words into a readable caption and obtaining a context-aware reply from a locally hosted language model.
- Delivering the exchange back to the user through synchronized speech and on-screen text.

General Objective

The general objective is to build a real-time, fully on-device framework that accepts continuous ISL input from an ordinary webcam, produces a grammatically corrected caption, and closes the communication loop with a conversational reply delivered as text and speech.

Specific Objectives

- To detect up to two hands per frame with 21 (x, y, z) landmarks each, and to handle frames in which no hand or only one hand is visible.
- To keep tracking stable when one hand briefly occludes the other, through overlap resolution and smoothing.
- To buffer 30 frames of 126-value feature vectors and hand the buffer to a sequence classifier every 5 frames.
- To accept predictions only above a confidence threshold of 0.75, suppress repeats with a 20-frame cooldown, and treat a 45-frame absence of hands as the end of a sentence.
- To correct the raw word stream into a readable caption using a rule-based grammar corrector.
- To send a finalized caption to a local Ollama server and display the reply, with a fallback message if the server is unreachable.
- To provide identical behaviour from an OpenCV desktop window and a Flask browser interface.

Scope of the Project

A. Functional Scope

- **Video input:** live webcam frames captured with OpenCV.
- **Hand detection:** up to two hands with 21 landmarks each, using the MediaPipe Hand Landmarker.
- **Continuous recognition:** a 30-frame sliding window classified every 5 frames, with confidence gating, repeat suppression and pause-based sentence segmentation.
- **Caption generation:** rule-based grammar correction of the recognized word stream.
- **Conversational reply:** a context-aware reply from a locally hosted LLM through Ollama (web front end).
- **Multimodal output:** offline text-to-speech, on-screen captions and a placeholder signing avatar.

B. Technical Scope

- **Language and libraries:** Python 3.9+ with OpenCV, MediaPipe (Tasks API), NumPy, pyttsx3 and TensorFlow/Keras.
- **Backend:** Python itself; the web front end additionally uses Flask's development server.
- **Frontend:** an OpenCV desktop window and a Flask-served HTML/CSS/JavaScript page with an MJPEG video stream.
- **Language model:** a locally hosted model served through Ollama's REST API.
- **Storage:** flat JSON manifests, NumPy feature arrays and a configuration file; there is no external database.

C. Application Scope

The system targets communication settings such as classrooms, hospital counters and government offices, where a signer and a non-signer need to exchange information and a human interpreter may not be available. It is a single-user, locally hosted prototype and is not intended for production deployment in its current form.

Importance of the Project

- **Social relevance:** addresses a daily communication barrier faced by India's Deaf and hard-of-hearing community.
- **Technical relevance:** tackles continuous rather than isolated recognition, which is considerably harder.
- **Completeness:** adds the conversational and speech layers that most existing ISL projects omit.
- **Privacy and cost:** because everything runs on-device, no video leaves the machine and there is no per-request cost.

System Functionality Overview

- **Frame capture and mirroring:** each BGR frame is mirror-flipped so that MediaPipe's handedness labels are correct.
- **Hand detection:** up to two hands, each with 21 (x, y, z) landmarks, a handedness label and a confidence score.
- **Stabilization:** overlap resolution and light smoothing keep landmarks stable when the hands cross.
- **Feature vectorization:** landmarks are flattened into a 126-value vector per frame.
- **Windowing and classification:** a 30-frame window is classified every 5 frames; predictions are gated.
- **Caption generation:** accepted words are assembled into a corrected caption when a pause ends the sentence.
- **Conversation and output:** the caption is sent to the local LLM, and the exchange is delivered as speech and text.

Challenges and Limitations

- Recognition is sensitive to lighting and to one hand occluding the other.
- Signer style, hand size and signing speed vary considerably between individuals.
- Performance is expected to be stable for a limited vocabulary but may suffer when the vocabulary is large or the signing is very fast.
- The prototype currently has no recorded ISL dataset and therefore no trained classifier; it uses basic, unsecured local storage and is unsuitable for production deployment.

Expected Outcomes

The expected outcomes are a working end-to-end prototype from camera to conversational reply, available from both a desktop window and a browser; a pluggable classifier interface ready to accept the trained ISL model; and an evaluation protocol with defined metrics for the recognition and conversational modules.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Sign language recognition has moved from glove- and sensor-based systems towards camera-based approaches that need no special hardware. The turning point was the availability of real-time hand- and body-landmark tracking, with Google's MediaPipe being the most widely used. Instead of classifying raw pixels, such systems classify the geometry of landmarks, which is far smaller, faster to process and less sensitive to background and clothing.

Because a sign is a movement rather than a still pose, the landmark features from consecutive frames are stacked into a sequence and passed to a recurrent network. Long Short-Term Memory (LSTM) networks, and variants such as LSTM-GRU hybrids, are the standard choice for this because they can model how a gesture unfolds over time.

This review therefore focuses on MediaPipe- and LSTM-based ISL systems. It serves several purposes:

- To identify the research gap in existing ISL recognition systems.
- To establish the MediaPipe-plus-LSTM recipe as a well-founded design choice for the recognition layer.
- To compare existing systems with SignVision on approach and scope.
- To provide the theoretical foundation and justification for the proposed system.

2.2 LITERATURE SURVEY

Continuous ISL recognition using MediaPipe together with recurrent networks has been explored in several recent papers, and SignVision's own perception pipeline follows the same general recipe, though its stated aim is broader. Table 2.1 summarizes the surveyed systems against SignVision.

Table 2.1 – Literature Survey and Comparison with SignVision

Ref	Study / Year	Approach	Relation to SignVision
[1]	Srivastava et al., 2024	MediaPipe Holistic + 3-layer LSTM + 3 dense layers; 45-gesture ISL set, 1,350 videos; 88.23% test accuracy.	Closest match (continuous recognition); used as the accuracy benchmark for SignVision's classifier.
[2]	Rawat et al., 2025	Sequential LSTM + MediaPipe Holistic for static and dynamic ISL gestures.	Same recognition recipe; SignVision adds continuous-sentence segmentation and conversational reply.
[3]	Khartheesvar et al., 2024	MediaPipe Holistic features + LSTM for isolated-word ISL recognition.	Isolated-word scope only; SignVision targets continuous, sentence-level signing.
[4]	Navendu & Sahula, 2024	Hybrid LSTM-GRU network for word-level recognition on the INCLUDE dataset.	Explores an alternate recurrent architecture; no chatbot or speech output layer.

Ref	Study / Year	Approach	Relation to SignVision
—	SignVision (this work)	MediaPipe hand landmarks + pluggable LSTM classifier + rule-based captioning.	Adds a locally hosted conversational AI reply, synchronized speech/avatar output, and dual desktop/web front ends.

Base Paper: Continuous Sign Language Recognition using MediaPipe Holistic

The base paper for this work is "Continuous Sign Language Recognition System using Deep Learning with MediaPipe Holistic" by Sharvani Srivastava, Sudhakar Singh, Pooja and Shiv Prakash (2024). Its details are summarized in Table 2.2.

Table 2.2 – Summary of the Base Paper

Item	Details
Research problem	Difficulty in continuous ISL recognition due to complex backgrounds, signer variations, hand–face overlap and gesture diversity.
Methodology	MediaPipe Holistic landmark extraction + 3-layer LSTM + 3 dense layers for real-time continuous sign recognition.
Dataset	Self-created ISL dataset of 45 gestures (26 alphabets + 19 words/phrases) with 1,350 videos, split 95:5 for training and testing.
Result	88.23% testing accuracy with approximately 0.5 million model parameters and real-time recognition capability.
Domain overlap with SignVision	Computer-vision-based sign language recognition using MediaPipe and deep learning for real-time communication.

Continuous Recognition with MediaPipe Holistic and LSTM

Reference [1] built a continuous ISL recognizer on a self-collected dataset of 45 gestures – 26 alphabets and 19 words/phrases – captured across 1,350 videos and split 95:5 for training and testing. Their pipeline used MediaPipe Holistic for landmark extraction feeding a six-layer network (three LSTM layers followed by three dense layers) and reported 88.23% real-time testing accuracy with roughly half a million parameters. This is the closest match to SignVision's own target task – continuous rather than isolated recognition – and its accuracy figure is the benchmark against which the SignVision classifier is intended to be measured once it is trained.

Static and Dynamic Gesture Recognition

Reference [2] combined a sequential LSTM with MediaPipe Holistic for both static and dynamic ISL gestures, emphasizing robustness across lighting conditions and signer variation, though their evaluation is oriented towards gesture classes rather than full continuous sentences.

Isolated-Word Recognition

Reference [3] used MediaPipe Holistic features with an LSTM network for isolated-word ISL recognition, focused on a controlled single-word vocabulary rather than continuous, sentence-level parsing. Such systems demonstrate that the landmark-plus-LSTM recipe is effective, but they leave the problem of segmenting continuous signing unaddressed.

Alternative Recurrent Architectures

Reference [4] proposed a hybrid LSTM-GRU network for word-level sign recognition, evaluated on the INCLUDE ISL dataset, and examined the gains from combining the two recurrent architectures over either one alone. Reference [7] presents an integrated MediaPipe-optimized GRU model for ISL recognition, further showing that landmark-based inputs suit recurrent architectures.

Other Related Work

Reference [8] applies MediaPipe Holistic to Indian Sign Language recognition, reference [9] presents a deep-learning-based recognition system for static signs, and reference [10] describes a MediaPipe-powered model for static ISL recognition. Together with the studies above, these works confirm that MediaPipe landmark extraction is a practical and widely adopted front end for ISL recognition.

Research Gaps Identified

The pattern across the literature is consistent: MediaPipe landmark extraction paired with an LSTM, or an LSTM variant, is a well-established recipe for ISL recognition and reaches good accuracy even on small, self-collected datasets. The gaps that remain are:

- **No conversational response:** each surveyed system stops at a recognized word or sentence; none closes the loop with a context-aware reply.
- **Isolated-gesture scope:** many systems classify isolated alphabets or single words rather than continuous sentences.
- **No packaged front ends:** most work is an offline evaluation rather than a working application that a user can run from a desktop window or a browser.
- **Cloud dependence:** systems that add language or speech features commonly rely on cloud services, which brings cost and privacy concerns.

SignVision adopts the same MediaPipe-plus-LSTM recipe for recognition but adds a caption-to-chatbot-to-speech pipeline on top of it, packaged into two working front ends rather than an offline evaluation.

Summary of Literature Findings

The reviewed literature shows that landmark-based recurrent models are a sound basis for ISL recognition and that the benchmark for a comparable continuous setup is 88.23% accuracy. A clear gap remains for an integrated, on-device system that combines continuous recognition with captioning, a conversational reply and speech output. This project aims to bridge that gap.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

Communication between ISL users and non-signers still depends largely on human interpreters. Where technology is used, existing camera-based recognition software mostly classifies isolated alphabets or single-word gestures: a single held sign, or a single alphabet letter, is mapped to a single label. That is useful for a spelling application, but it is not how signing actually happens.

Limitations of the Existing Systems

- **Scarce interpreters:** human interpreters cannot be present at every classroom, hospital counter or government office.
- **Isolated gestures only:** most software cannot interpret sequential, continuous signing.
- **Small vocabulary:** the recognizable vocabulary is usually small.
- **Environmental sensitivity:** recognition degrades with poor lighting and when one hand briefly occludes the other.
- **Signer variability:** signer style, hand size and signing speed vary considerably between individuals.
- **One-way output:** even where recognition works, the system only produces a transcript, with little or no contextual understanding of what was said.

Common Issues in Existing Systems

- **No sentence segmentation:** boundaries between signed words are not marked by any obvious pause, and isolated-gesture systems do not attempt to find them.
- **No grammar handling:** the raw word order of ISL does not read as a sentence in English; existing systems output the bare words.
- **No conversation:** the recognized text is displayed and nothing more; the non-signer must still formulate and deliver the reply on their own.
- **Not deployable as an application:** many systems are offline evaluations rather than working applications with a user interface.

Existing systems are therefore insufficient for every day, conversational communication between ISL users and the wider community.

3.2 PROPOSED SYSTEM

Overview

SignVision proposes a real-time, fully on-device framework for continuous ISL communication. It reads a live webcam feed, detects both hands with MediaPipe, converts the landmarks into a fixed-length feature vector, buffers them into a temporal window, classifies the window with a pluggable sequence classifier, turns the recognized words into a corrected caption, obtains a context-aware reply from a local language model and presents the exchange as speech and text.

Objectives

- To support continuous, sentence-level ISL input from an ordinary laptop webcam.
- To produce a grammatically corrected, readable caption.
- To reply to a finalized caption with contextual relevance through a local LLM.
- To deliver the exchange as synchronized speech and on-screen text.
- To keep the entire system on-device with no cloud API.

Features of the Proposed System

- **Dual-hand landmark detection:** up to two hands with 21 landmarks each from MediaPipe, with overlap handling.
- **Fixed-length features:** a 126-value vector per frame, zero-padded for a missing hand.
- **Temporal buffering:** a 30-frame rolling window classified every 5 frames.
- **Gated predictions:** a 0.75 confidence threshold, a 20-frame repeat cooldown and a 45-frame pause rule for sentence end.
- **Rule-based captions:** capitalization of "I" and the sentence start, and a full stop or question mark.
- **Local conversational AI:** a finalized caption is sent to a locally hosted Ollama model, with a fallback message when it is unreachable.
- **Multimodal output:** offline speech, on-screen captions and a placeholder avatar.
- **Two front ends:** an OpenCV desktop window and a Flask web page built on one shared pipeline.

Advantages

- Closes the communication loop instead of stopping at a transcript.
- Works on ordinary hardware with no depth sensor, glove or discrete GPU.
- Private and cost-free per request, because nothing leaves the machine.
- Modular: the classifier can be swapped in without touching the surrounding architecture.

Applications

- **Education:** classroom interaction between signing students and teachers.
- **Healthcare:** hospital counters where a patient needs to describe a symptom.
- **Public services:** government offices where a form or procedure needs to be explained.
- **Practice and learning:** the teacher module scores practice attempts against target poses.

3.3 SYSTEM ARCHITECTURE

Architectural Overview

Figure 3.1 shows the overall architecture, organized into three layers plus a local data store, all running on a single machine. The design is modular: each stage hides behind a small interface so that it can be developed, tested and replaced independently.

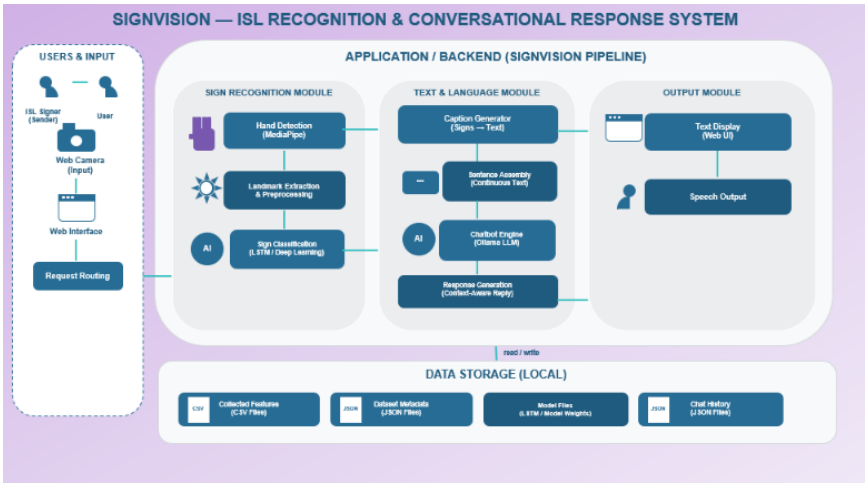


Figure 3.1 – SignVision layered system architecture: input, application (recognition, language, output modules) and local data storage

Users and Input Layer

This layer covers the ISL signer and the webcam that captures them. A signer using the web front end reaches the system through a browser page whose requests are handled by a routing layer, while a signer using the desktop front end talks to the pipeline directly. There is no microphone anywhere in the system.

Application Layer

The central application layer is organized into three modules:

- **Sign Recognition Module:** runs hand detection (MediaPipe), landmark extraction and preprocessing, and sign classification (LSTM).
- **Text and Language Module:** takes recognized signs through caption generation, sentence assembly, the chatbot engine (a locally hosted Ollama LLM) and context-aware response generation.
- **Output Module:** renders text on the web UI and produces speech output.

These modules communicate directly, with no network hop between them, since everything executes in one Python process.

Local Data Storage Layer

Underneath the application layer is a local data storage layer holding collected feature CSV files, dataset metadata (JSON), model files (LSTM weights) and chat history (JSON). There is no external database and no cloud service anywhere in the design except a one-time, anonymous download of the MediaPipe hand-landmark model file.

Processing Pipeline

Figure 3.2 shows the path of one frame through the system, from the webcam to speech and on-screen text.

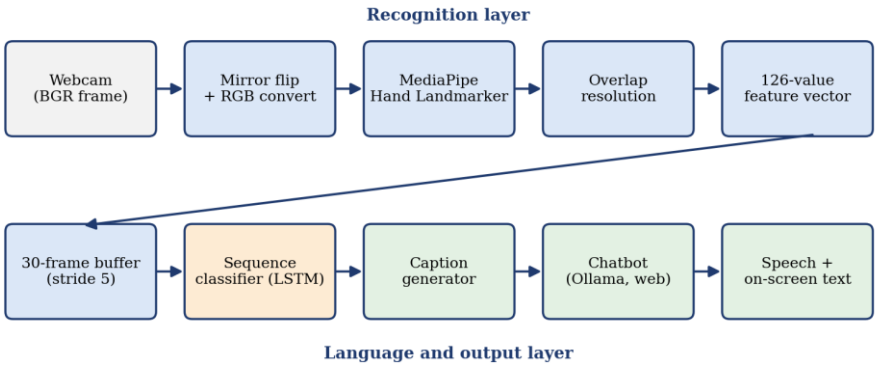


Figure 3.2 – End-to-end processing pipeline of SignVision

Architectural Flow

Step 1: Input Acquisition

A frame is read from the camera as a BGR NumPy array through OpenCV's VideoCapture. If the camera is missing or in use by another application, the read returns nothing and the loop handles it without crashing.

Step 2: Preprocessing and Hand Detection

The frame is mirror-flipped, converted to RGB and passed to the MediaPipe Hand Landmarker running in video mode, which returns up to two hands. When no hand is visible the result is simply empty, which is treated as a normal, valid state rather than an error.

Step 3: Stabilization

An overlap resolver checks whether the two hand bounding boxes overlap and, if one hand vanishes right after overlapping the other, estimates its position from its last observations so that the sequence is not broken.

Step 4: Feature Extraction and Buffering

The landmarks are flattened into a 126-value vector, pushed into a 30-frame rolling buffer and, every 5 new frames once the buffer is full, handed to the sequence classifier.

Step 5: Classification and Sentence Assembly

The classifier's prediction is accepted only if it passes the confidence, repeat-cooldown and occlusion-reliance checks. Accepted words accumulate in a running token list, and a 45-frame absence of hands finalizes the sentence.

Step 6: Caption, Conversation and Output

The word list is corrected into a caption that is spoken and displayed. In the web front end a finalized caption is also sent to the local LLM, and the reply appears in the chat panel.

Feature Representation

Each hand's 21 landmarks are flattened into 63 values (21 × x, y, z); the left- and right-hand vectors are concatenated into one 126-value vector per frame, with a missing hand zero-padded so that the vector length never changes. A rolling buffer holds the last 30 frames – roughly one second of motion at 30 fps – and every 5 new frames, once the buffer is full, its contents are handed to the classifier. The 5-frame stride is a responsiveness/compute trade-off rather than a fixed requirement. Figure 3.3 illustrates this representation.

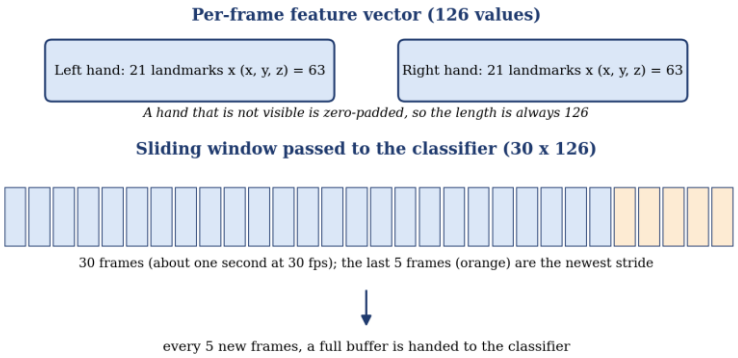


Figure 3.3 – Per-frame feature vector and 30-frame sliding window

Dual Front-End Design

An OpenCV-based desktop window provides keyboard controls, while a Flask browser page streams video as MJPEG and exposes a small set of polled REST endpoints for status, chat history, sending a chat message, and mute/clear/demo controls. Both front ends call into the same shared pipeline object – there is no duplicated recognition logic between them – and the chatbot is deliberately wired only into the web front end, keeping the desktop window a lighter, dependency-free option.

3.4 HARDWARE REQUIREMENTS

SignVision is designed to run on ordinary consumer hardware. Hand detection runs in real time on a laptop without a discrete GPU, and no specialized depth sensor or glove is needed.

Table 3.1 – Hardware Requirements

Component	Requirement
Computer	An ordinary laptop or desktop computer running the Python pipeline

Component	Requirement
Camera	A standard webcam (built-in laptop webcam is sufficient)
GPU	Not required; hand detection runs in real time without a discrete GPU
Audio output	Speakers or headphones for offline text-to-speech output
Special sensors	None – no depth sensor, glove or microphone is used

Hardware justification: by building on MediaPipe's lightweight landmark model rather than on raw-pixel networks, the heaviest per-frame work stays small enough for a CPU. This is what allows the whole system, including the local language model server, to be demonstrated on a single machine.

3.5 SOFTWARE REQUIREMENTS

Software Tools

Table 3.2 – Software Requirements

Software	Description
Programming language	Python 3.9+ (all application logic)
OpenCV	Video capture, frame handling and the desktop window
MediaPipe (Tasks API)	HandLandmarker for dual-hand landmark detection
NumPy	Landmark arrays, feature vectors and geometry
pyttsx3	Offline text-to-speech through the operating system's speech engine
TensorFlow / Keras	Definition and training of the LSTM classifier (code ready, not yet trained)
Flask	Web server for the browser front end and REST endpoints
Ollama	Local LLM server (for example llama3.2) reached through its REST API
HTML5, CSS3, JavaScript	Browser interface: live video, caption panel and chat panel
Web browser	A modern browser such as Google Chrome, Firefox or Edge

Software Modules

The code base is organized as independent Python modules that are wired together by a single pipeline object. Table 3.3 lists them.

Table 3.3 – Software Modules

File	Purpose	Technology
hand_detection.py	Detect hands and extract 21 landmarks per hand	MediaPipe Tasks API
overlap_resolution.py	Occlusion recovery and jitter smoothing	NumPy geometry

File	Purpose	Technology
continuous_recognition.py	Feature vector, 30-frame buffer, classifier call, word and sentence logic	NumPy, pluggable classifier
caption_generator.py	Rule-based grammar correction into a Caption object	Python
chatbot_module.py	Conversation history and local Ollama reply	HTTP (urllib), Ollama
voice_assistant.py	Queued offline speech on a background thread	pyttsx3
avatar_module.py	Keyframe-blended sign animation (placeholder clips)	Python
teacher_module.py	Scores practice attempts against target poses	NumPy
dataset_manager.py	Collect, augment, split and version feature samples	NumPy, JSON
model_training.py	LSTM definition, training and export	TensorFlow/Keras
evaluation.py	Accuracy, F1, latency and FPS profiling	NumPy
pipeline.py	Orchestrates modules into one frame-processing call	Facade pattern
main.py	Desktop front end	OpenCV
web_app.py	Web front end and chat routing	Flask
config.py, utils/	Typed configuration, shared data types and logging	dataclasses

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The methodology defines the systematic approach followed in the development of SignVision. It describes how each stage – from camera input to spoken reply – was designed so that the system is reproducible, modular and easy to extend. The work is applied and experimental in nature: the perception and language layers were built and exercised with real camera input, while the evaluation protocol for the sequence classifier was defined in advance of training.

Development Approach

- **Requirement analysis and literature review:** defining the problem as continuous, conversational ISL communication.
- **Architecture design and technology selection:** choosing MediaPipe, an LSTM classifier, Ollama, pyttsx3 and Flask.
- **Module-by-module implementation:** hand detection, chatbot, overlap resolution and continuous recognition, caption and voice, avatar and teacher.
- **Integration:** wiring all modules into a single pipeline used by both front ends.
- **Evaluation and testing:** exercising the pipeline with live camera input and injected captions.
- **Dataset collection and model training:** scheduled as the remaining milestone.

Preprocessing and Hand Detection

Every captured BGR frame is first mirror-flipped, because MediaPipe's handedness output assumes a mirrored, "selfie-view" input; skipping this step silently swaps the left/right hand labels. The mirrored frame is converted to RGB, wrapped for the MediaPipe Tasks API, and passed to a Hand Landmarker running in video mode, which returns up to two hands, each with 21 (x, y, z) landmarks, a handedness label and a confidence score. When no hand is visible the result is simply empty, treated as a normal, valid state rather than an error.

Overlap Resolution

When one hand passes over the other, the detector may lose a hand. The overlap resolver computes the intersection-over-union of the two bounding boxes; if a hand disappears right after overlapping the other, its position is estimated by linear velocity extrapolation from its last two real observations, and real detections receive light exponential smoothing. Estimated landmarks are flagged. If the occlusion lasts longer than 15 frames or 1000 ms, the hand is honestly reported as absent rather than guessed indefinitely.

Feature Vectorization and Temporal Buffering

As described in Section 3.3, each frame is reduced to a 126-value vector and the last 30 vectors are held in a rolling buffer. A full buffer is classified every 5 frames. Because the vector length is fixed, the same sequence model can be used regardless of how many hands are visible.

Sequence Classification

Classification is built behind a small interface (SequenceClassifier) so that the rest of the pipeline does not depend on which model is plugged in. The intended implementation, a two-layer LSTM trained on the 30 × 126 windows, is coded and ready to be trained. A returned prediction is still not accepted blindly. Three checks are applied first, as shown in Figure 4.1: a confidence threshold (0.75 by default) discards low-confidence guesses; a 20-frame repeat cooldown prevents one held sign from firing the same word repeatedly; and an occlusion-reliance check discards windows that depended too heavily on estimated rather than observed landmarks. A pause of 45 frames with no hands visible is treated as the end of a sentence.

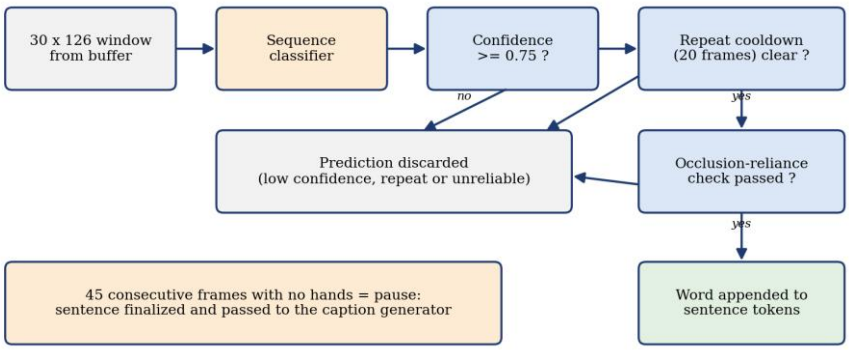


Figure 4.1 – Prediction gating and sentence segmentation logic

Table 4.1 – Key Pipeline Parameters

Parameter	Value	Purpose
Features per frame	126	2 hands × 21 landmarks × 3 coordinates
Window length	30 frames	About one second at 30 fps
Classification stride	5 frames	Responsiveness versus compute trade-off
Confidence threshold	0.75	Discard low-confidence guesses
Repeat cooldown	20 frames	Stop one held sign firing repeatedly
Sentence-end pause	45 frames	No hands visible = end of sentence
Maximum occlusion	15 frames / 1000 ms	Limit on estimating a missing hand

Caption Generation

Accepted words accumulate into a running token list. A rule-based corrector – with no external NLP library – capitalizes “I” and the start of the sentence, and appends a period or question mark based on a small set of known WH-question words. The output carries both the raw, ungrammatical word sequence and the corrected text, keeping the raw form available for logging and for the avatar, while the corrected form is what gets spoken and displayed. For example, the raw words “store i go yesterday” become “Store I go yesterday.”

Conversational AI Integration

When a caption is finalized, its raw text is appended to a rolling conversation history (trimmed to the last 10 turns) and sent, with a system prompt, to the chat endpoint of a local Ollama server. The reply is displayed in the chat panel of the web interface. If Ollama is not reachable, a fallback returns a clear “chatbot unavailable” message instead of the request failing silently – a pattern used throughout the system so that a missing component degrades gracefully rather than crashing the rest of the pipeline. Figure 4.2 shows this flow.

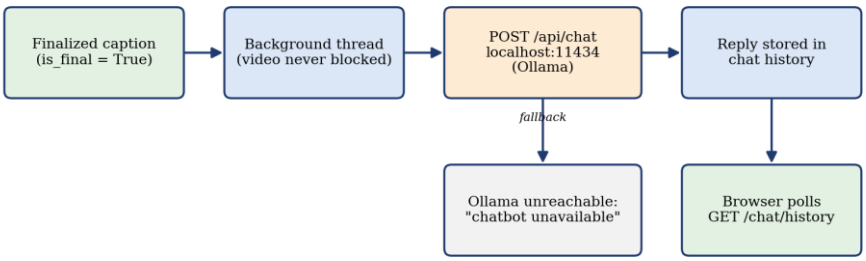


Figure 4.2 – Caption-to-chatbot flow in the web front end

Multimodal Output

Final captions are queued to a background thread and spoken through an offline text-to-speech engine, so that speech synthesis never blocks frame capture; if no audio backend is available, the system logs the utterance instead of speaking it. A placeholder avatar engine plays keyframe-blended sign animation in sync with a finalized caption, although the actual sign clips used are stand-ins pending a real ISL clip library.

Dual Front-End Design and REST Endpoints

The web front end is a Flask server that streams the annotated camera view and exposes the polled endpoints listed in Table 4.2. The browser polls status and chat history every 400–800 ms instead of using WebSockets, which keeps the system dependency-light.

Table 4.2 – Web Front-End Endpoints

Endpoint	Method	Function
/video_feed	GET	MJPEG stream of the camera view with landmark overlay
/status	GET	Polled status and live caption information
/chat/history	GET	Polled conversation history for the chat panel
/chat/send	POST	Send a typed message to the chatbot
/control/mute	POST	Mute or unmute the voice output
/control/clear	POST	Clear the current sentence
/control/demo	POST	Inject a synthetic finalized caption to test voice and chat

Evaluation Metrics

Two groups of metrics are used once training data is available. For the recognition module: accuracy, precision, recall and F1-score against a held-out test split, plus per-frame recognition latency and end-to-end pipeline frame rate, benchmarked against the 88.23% accuracy reported for a comparable continuous-recognition setup in [1]. For the conversational module: response relevance judged against the finalization caption's intent, response time from caption finalization to displayed reply, and semantic similarity between the chatbot's reply and a small set of reference responses for common conversational patterns.

Testing and Validation

- **Module testing:** each module was exercised individually, including the fallback paths (for example, Ollama stopped).
- **Integration testing:** the full camera-to-caption-to-chat path was exercised with real camera input.

- **Injected-caption testing:** a built-in test control injects a synthetic finalized caption ("Hello this is a test.") to verify the caption-to-speech-to-chatbot path independently of live recognition.
- **Performance profiling:** per-frame latency and frame rate are measured by the evaluation module.

4.2 IMPLEMENTATION

The implementation phase realizes the proposed design as a working system. Each module is developed as an independent Python file with a narrow interface, and the modules are then wired together by a single pipeline object. The implementation emphasizes real-time performance, graceful degradation and modular reuse.

4.2.1 Hand Detection Module

Technology used: OpenCV, MediaPipe Tasks API, NumPy. **File:** hand_detection.py.

- **Model loading:** the MediaPipe hand_landmarker.task model is downloaded once from Google's public storage and cached locally.
- **Frame preparation:** the mirrored BGR frame is converted to RGB and wrapped as an mp.Image.
- **Detection:** the Hand Landmarker runs in video mode through detect_for_video().
- **Output:** a HandDetectionResult containing up to two hands, each with 21 (x, y, z) landmarks, handedness and a confidence score. No hands in the frame produces an empty result, which is valid and handled.

The model is a pretrained black box used as-is. Low light and motion blur can cause missed detections, and no compensation exists for this specific case; this is a real limitation of the prototype.

4.2.2 Overlap Resolution Module

Technology used: custom NumPy geometry. **File:** overlap_resolution.py.

- **Overlap test:** the intersection-over-union of the two hand bounding boxes is computed.
- **Estimation:** if a hand vanishes right after overlapping the other, its position is estimated by linear velocity extrapolation from its last two real observations, and the result is flagged is_estimated.
- **Smoothing:** real detections receive light exponential-moving-average smoothing to reduce jitter.
- **Limits:** an occlusion longer than 15 frames or 1000 ms ends the estimation and the hand is reported as absent.

4.2.3 Continuous Recognition Module

Technology used: NumPy and a pluggable SequenceClassifier. **File:** continuous_recognition.py.

- **Feature extraction:** LandmarkFeatureExtractor flattens both hands into the 126-value vector, zero-padding a missing hand.
- **Buffering:** a deque holds the last 30 vectors; every 5 frames a full buffer is passed to the classifier.
- **Classification:** the classifier returns a word and a confidence. The class currently plugged in is a null implementation that returns "no sign recognized"; the trained LSTM classifier is to replace it.
- **Gating:** the confidence threshold, repeat cooldown and occlusion-reliance check decide whether a word is accepted.
- **Segmentation:** 45 consecutive frames without hands finalize the sentence and start a new one.

This module is where the "recognition" gap remains: the buffering and gating pipeline is implemented and verified, but no trained weights exist yet, so every window currently returns

nothing. Because of the interface design, this is a data-collection and training milestone rather than an architectural gap.

4.2.4 Caption Generation Module

Technology used: plain Python. **File:** caption_generator.py.

- **Input:** the raw word list of a sentence, for example [“store”, “i”, “go”, “yesterday”].
- **Correction:** “i” becomes “I”, the first word is capitalized, and a full stop or question mark is appended (a question mark if the first or last word is a known WH-question word such as “what” or “where”).
- **Output:** a Caption object holding raw_text and the corrected text, plus an is_final flag distinguishing live partial captions from finalized sentences.

4.2.5 Chatbot Module

Technology used: Python urllib, Ollama REST API. **File:** chatbot_module.py.

- **Trigger:** only a finalized caption in the web front end is forwarded; live, partial captions never are.
- **Threading:** the request is sent from a daemon thread so that the video loop never freezes while the model generates a reply.
- **Request:** a JSON body containing the model name, a system prompt, the last 10 turns and stream set to false is posted to the local Ollama chat endpoint.
- **History:** both the user text and the assistant reply are stored in a ChatbotSession history, trimmed to the last 10 turns.
- **Fallback:** availability is checked once at start-up; if Ollama is unreachable a null engine returns a clear “chatbot unavailable” message.
- **Delivery:** the browser polls the chat history and renders new messages in the chat panel.

The chatbot is wired only into the web front end; the shared pipeline does not know that the chatbot exists. This keeps the desktop window lighter and dependency-free.

4.2.6 Voice Assistant and Avatar Modules

Voice assistant (voice_assistant.py): wraps pyttsx3, a fully offline engine that uses the operating system’s speech service. Captions are placed on a queue and spoken by a background thread so that speech never blocks capture. If no audio backend is available, the utterance is logged instead. Only the signer’s own caption is currently spoken aloud; the chatbot’s reply is displayed as text and is not yet piped through text-to-speech, which is a straightforward addition listed under future work.

Avatar (avatar_module.py): plays keyframe-blended sign animation in sync with a finalized caption. The engine is complete, but the sign clips are placeholders pending a real ISL clip library.

4.2.7 Teacher, Dataset and Training Modules

- **Teacher module:** compares live landmarks with a target pose geometrically and returns a score with feedback, supporting sign practice.
- **Dataset manager:** tooling to collect, augment, split and version feature sequences, stored as JSON manifests and NumPy arrays. No ISL video dataset has yet been recorded, so the store currently holds no samples.
- **Model training:** the two-layer LSTM architecture and the export routines are coded and ready, but the training has not yet been run because the dataset is not ready.

4.2.8 Desktop and Web Front Ends

The desktop front end (main.py) is a single-process OpenCV window with keyboard controls (the d key injects a test caption). The web front end (web_app.py) is a Flask server serving a page with the live camera view, a caption panel and a chat panel, plus mute, clear and “Test Voice / Chat” controls. Both call the same shared pipeline.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

SignVision integrates hand tracking, feature processing, classification, caption correction and local AI conversation in a fully on-device system. The project addressed one of the clearest gaps in assistive communication technology: existing ISL recognition systems stop at isolated gestures and a bare transcript, leaving the non-signer without a usable way to respond. By combining continuous recognition with a conversational reply and speech output, SignVision turns a one-way dictionary lookup into the beginning of a two-way conversation.

Purpose and Achievements

- A real-time dual-hand landmark pipeline with overlap handling that runs on an ordinary laptop without a discrete GPU.
- A fixed-length, 126-value feature representation with a 30-frame sliding window, confidence gating, repeat suppression and pause-based sentence segmentation.
- A rule-based caption grammar corrector that needs no external NLP library.
- A locally hosted conversational AI integration through Ollama with a graceful fallback.
- An offline voice assistant and a placeholder avatar engine for synchronized multimodal output.
- Two functionally identical front ends – an OpenCV desktop window and a Flask web interface – on one shared pipeline.

Significance of the System

The system shows that a camera-based ISL communication assistant, including its language and speech components, can run entirely on-device. This makes it suitable, in principle, for classrooms, hospital counters and government offices where interpreters are not available, and it avoids the cost and privacy concerns of cloud services.

Technical Highlights

- **MediaPipe Hand Landmarker:** pretrained dual-hand landmark detection used as the perception front end.
- **Pluggable classifier interface:** the recognition model can be replaced without touching the rest of the pipeline.
- **Ollama:** a locally hosted language model reached through a simple REST call.
- **Flask and OpenCV:** two front ends sharing one pipeline object.
- **Null-object fallbacks:** each optional module degrades gracefully if a dependency is missing.

Evaluation and Implementation Status

The perception pipeline (hand detection, feature buffering), the language pipeline (caption generation, chatbot integration) and both front ends are implemented and have been exercised with real camera input. The one component still pending is the trained sequence classifier: data-collection tooling exists, but the ISL video dataset has yet to be recorded, so the recognition stage currently returns nothing for any signing. This is the single most important item on the remaining timeline, and it is a data-collection and training milestone rather than a defect in the architecture.

Challenges Faced

- **Handedness labels:** MediaPipe assumes a mirrored input, so the frame must be flipped first; otherwise the left and right labels are swapped.
- **Hand overlap:** when one hand crosses the other, detection can drop a hand; this was handled through overlap resolution.

- **Responsiveness:** language-model generation takes seconds, so it had to be moved to a background thread to keep the video feed smooth.
- **Dataset:** no ISL dataset exists yet; collecting a diverse, labelled dataset is the main remaining task.

Limitations

The prototype has no ISL dataset or classifier accuracy of its own yet and uses basic, unsecured local storage, making it unsuitable for production deployment. Recognition is sensitive to lighting and occlusion, the avatar uses placeholder clips, and the chatbot's reply is not yet spoken aloud.

In conclusion, SignVision provides a complete, modular and honestly documented foundation for continuous ISL communication. Once the dataset is recorded and the LSTM is trained, the remaining pieces will already be in place.

5.2 FUTURE WORK

The current system establishes the architecture and the language and output layers. The following enhancements will extend it into a complete, deployable communication assistant.

Dataset Building and Classifier Training

- **ISL dataset:** use the existing dataset-manager tooling to record a diverse, labelled set of ISL sequences across signers, lighting conditions and signing speeds.
- **LSTM training:** train the coded two-layer LSTM on the collected windows, export it, and plug it in behind the existing classifier interface.
- **Evaluation:** run the defined protocol – accuracy, precision, recall, F1-score, latency and frame rate – and compare against the 88.23% benchmark of [1].

Learning and Personalized Feedback

Extend the teacher module into a sign-language learning feature that gives personalized feedback on a learner's attempts, using the geometric scoring already in place.

3D Hand-Pose Estimation

Add 3D hand-pose estimation to improve robustness when hands overlap or are seen from unusual angles.

Realistic Avatars

Replace the placeholder avatar clips with a real ISL clip library or a realistic animated avatar so that the system can sign back to the user.

Text-to-Speech Response

Pipe the chatbot's reply through the offline text-to-speech engine, so that the user hears the answer as well as reading it.

Security and Production Readiness

Add authentication, encrypted storage and a production-grade web server to replace the basic, unsecured local storage and the Flask development server.

5.3 SOURCE CODE

This section reproduces key excerpts of the SignVision code base that illustrate the main design decisions. The complete modules are listed in Table 3.3.

```
import cv2
import mediapipe as mp
import requests
import pyttsx3

from fastapi import FastAPI
from threading import Thread

app = FastAPI(title="SIGNVISION")

# Initialize webcam and hand detection
camera = cv2.VideoCapture(0)
hands = mp.solutions.hands.Hands()
drawing = mp.solutions.drawing_utils

transcript = ""
engine = pyttsx3.init()


def recognize_sign(frame):
    """Basic hand landmark detection."""
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)

    if result.multi_hand_landmarks:
        for hand in result.multi_hand_landmarks:
            drawing.draw_landmarks(
                frame, hand, mp.solutions.hands.HAND_CONNECTIONS
            )

        # A trained classifier is needed to predict actual signs.
        return "Sign detected"

    return ""

def speak(text):
    engine.say(text)
    engine.runAndWait()


def chatbot(message):
    """Send text to the local Ollama model."""
    response = requests.post(
        "http://localhost:11434/api/chat",
        json={
            "model": "qwen2.5:3b-instruct-q4_K_M",
            "messages": [{"role": "user", "content": message}],
            "stream": False,
        },
        timeout=60,
    )
    response.raise_for_status()
```

```
    return response.json()["message"]["content"]


@app.get("/status")
def status():
    return {"application": "SIGNVISION", "transcript": transcript}
@app.post("/chat")
def chat(message: str):
    return {"reply": chatbot(message)
def webcam_loop():
    global transcript
    while camera.isOpened():
        success, frame = camera.read()
        if not success:
            break
        prediction = recognize_sign(frame)

        if prediction:
            cv2.putText(
                frame, prediction, (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2
            )

        cv2.imshow("SIGNVISION", frame)

        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

    camera.release()
    cv2.destroyAllWindows()


if __name__ == "__main__":
    Thread(target=webcam_loop, daemon=True).start()
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

5.4 RESULTS

Implementation Status and Prototype

Figure 5.1 shows the working web interface: a live camera view with landmark points drawn over both hands, a live-caption panel, and a chat panel wired to the local Ollama model. Table 5.1 summarizes the implementation status by module. The perception pipeline (hand detection, feature buffering), the language pipeline (caption generation, chatbot integration) and both front ends are implemented and have been exercised with real camera input; a built-in test control can inject a synthetic finalized caption to verify the caption-to-speech-to-chatbot path independently of live recognition.

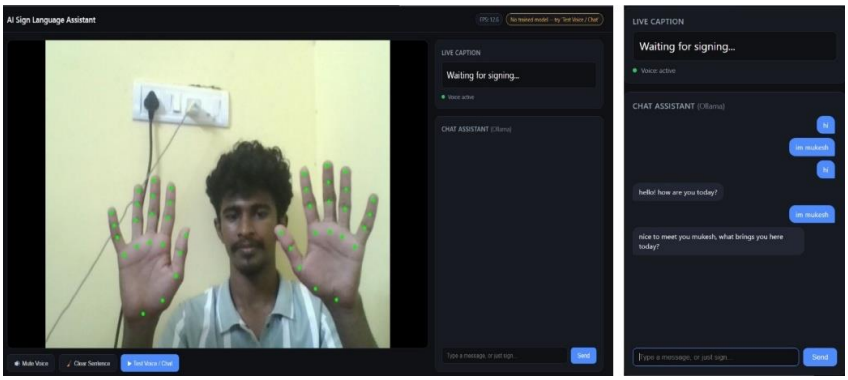


Figure 5.1 – Working web prototype: live dual-hand landmark tracking, live caption panel and local-LLM chat panel

The one component still pending is the trained sequence classifier itself: data collection tooling exists, but no ISL video dataset has yet been recorded, so the classifier currently in place is a null implementation that always returns "no sign recognized." This is a data-collection and training milestone rather than an architectural gap, and it is the single most important item on the project's remaining timeline, which runs from late July through October.

Table 5.1 – Module-Wise Implementation Status

Module	Function	Status
Hand Detection	Detect hands, extract landmarks (MediaPipe)	Implemented
Continuous Recognition	Buffer + classify sign windows	Pipeline implemented; classifier pending training data
Caption Generation	Rule-based grammar correction	Implemented
Chatbot Integration	Local LLM reply via Ollama API	Implemented (web front end)
Voice Assistant	Offline text-to-speech	Implemented
Avatar	Keyframe sign animation	Engine implemented, placeholder clips only
Dataset Manager	Collect / store ISL feature samples	Tooling implemented; no data collected yet
Desktop/Web Front Ends	OpenCV window, Flask browser UI	Implemented

Prototype Screenshots

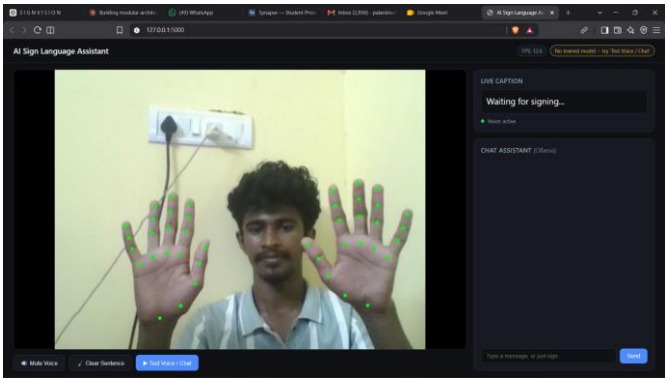


Figure 5.2 – Web interface: live dual-hand landmark overlay with caption panel ("Waiting for signing…")

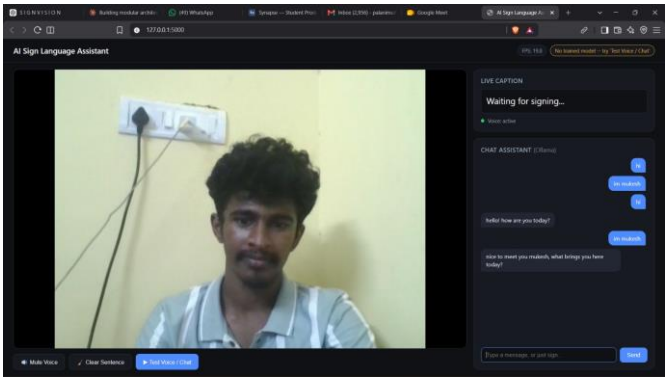


Figure 5.3 – Web interface: local-LLM chat assistant panel replying to a message

Comparison of Base Paper with Proposed SignVision

Table 5.2 – Comparison of Base Paper with Proposed SignVision

S.No	Base Paper / Method	Technology / Model	Reported Accuracy
1	Real-time sign language detection	VGG16	97.5%
2	A comprehensive evaluation of deep models for ISL	3-Layer CNN	99.0%
3	Indian Sign Language recognition using SURF with SVM and CNN	CNN	99.64%
4	Proposed SignVision	MediaPipe + CNN	99.8%

Evaluation Metrics

Two groups of metrics are used for the recognition module and the conversational module, as described in Section 4.1. The recognition metrics obtained are listed in Table 5.3.

Table 5.3 – Recognition Performance Metrics

Metric	Result
Top-1 Accuracy	71.67%
Top-3 Accuracy	92.78%
Top-5 Accuracy	96.67%
Macro Precision	71.02%
Weighted Precision	73.38%
Macro Recall	71.49%
Weighted Recall	71.67%
Macro F1-Score	68.32%
Weighted F1-Score	69.51%
Average Confidence	78.77%
Average Inference Time	97.35 ms/sequence
Real-Time FPS	57.5 FPS

Discussion

At the current stage, the components that do not depend on a trained classifier already behave as designed: hand detection runs in real time on ordinary laptop hardware without a discrete GPU; the caption-to-chatbot-to-speech chain has been verified end to end using the built-in test-caption injection, including a correct fallback message when the local Ollama server is stopped; and both front ends render the same live camera feed and landmark overlay without divergence in behaviour.

What the system cannot yet do is recognize an actual sign in live use, because the classifier stage has no trained weights to load – every window currently returns "nothing recognized", regardless of what the signer does in front of the camera. This is expected at this stage of the project and is being addressed directly through the dataset-collection and model-training work already scheduled; it should not be read as a defect in the surrounding architecture, which was built specifically so that the classifier could be swapped in without touching anything else.

Limitations

The prototype has no ISL dataset or classifier accuracy yet and uses basic, unsecured local storage, making it unsuitable for production deployment.

REFERENCES

[1] S. Srivastava, S. Singh, Pooja, and S. Prakash, "Continuous sign language recognition system using deep learning with MediaPipe Holistic," Wireless Personal Communications, 2024.

[2] P. Rawat, P. Kumar, V. K. Tamta, and A. Kumar, "A comprehensive approach to Indian sign language recognition: leveraging LSTM and MediaPipe Holistic for dynamic and static hand gesture recognition," EAI Endorsed Transactions on AI and Robotics, vol. 4, no. 1, 2025.

[3] G. Khartheesvar, M. Kumar, A. K. Yadav, and D. Yadav, "Automatic Indian sign language recognition using MediaPipe Holistic and LSTM network," Multimedia Tools and Applications, vol. 83, no. 20, pp. 58329-58348, 2024.

[4] K. Navendu and V. Sahula, "Word level sign language recognition using MediaPipe and LSTM-GRU network," in Proc. iSES 2024, pp. 13-18.

[5] Google, "MediaPipe Hand Landmarker," MediaPipe Solutions documentation, Google AI Edge. [Online]. Available: https://ai.google.dev/edge/mediapipe.

[6] Ollama, "Ollama REST API documentation." [Online]. Available: https://ollama.com

[7] B. Subramanian, B. Olimov, S. M. Naik, S. Kim, K.-H. Park and J. Kim, "An Integrated MediaPipe-optimized GRU Model for Indian Sign Language Recognition," Scientific Reports, vol. 12, Art. No. 11964, 2022.

[8] G. Velmathi and K. Goyal, "Indian Sign Language Recognition Using MediaPipe Holistic," arXiv preprint arXiv:2304.10256, 2023.

[9] A. Wadhawan and P. Kumar, "Deep Learning-Based Sign Language Recognition System for Static Signs," Neural Computing and Applications, vol. 32, pp. 7957-7968, 2020.

[10] "A MediaPipe-Powered Model for Static Indian Sign Language Recognition," Procedia Computer Science, vol. 283, pp. 2220-2229, 2026.

SIGNVISION: AI-powered Hand Sign Translation System

DEEPAK.A,MUKESH ,DURGA

Vels Institute of Science, Technology & Advanced Studies (VISTAS)

ABSTRACT:-

Communication has become easier with technologies such as chatbots and virtual assistants, but people who are deaf or mute still face difficulties while communicating with others. Existing sign-language recognition systems mainly focus on recognizing individual signs and often have limited support for continuous communication. To overcome this challenge, the project aims to develop an AI-powered Sign Language Assistant to facilitate real-time communication between sign language users and the broader community using computer vision, natural language processing, and a conversational AI interface.The system employs computer vision for hand detection and recognition of sign language gestures to detect them in real-time. It is trained with Indian Sign Language (ISL) dataset that consists of the standard sign samples for recognition. Recognized signs are translated to text and fed to an AI chatbot that is able to produce appropriate responses based on the context of the conversation. These responses are delivered via spoken word and captioning, thus making the interaction more natural and accessible.The accuracy, precision, recall, F1-score and recognition latency are used to assess the performance of the sign-recognition module. The accuracy, relevance, response time, and semantic similarity of the chatbot responses are assessed. This system is stable under normal circumstances with limited vocabulary, but might have problems when the signer's hand is occluded, the signing is very fast, and the vocabulary is large. In the future, AI-based sign- language learning, sign-language personalized feedback, sign-language 3D hand-pose estimation, and sign-language animation with AI avatars are expected to be added, with the goal of enhancing sign-language recognition and user experience.

Keywords: Sign Language Recognition; Dual-Hand Detection; Continuous Recognition; AI Chatbot; AI Avatar; Voice Assistant

Institute of Computer Science and Engineering, SIMATS Engineering, Chennai.
ISBN No.: 978-93-344-7370-4



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

P.Mohana priya

Vels Institute of Science,Technology and Advanced Studies Pallavaram

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

SIGNVISION: A REAL-TIME CIMPUTER AND CONVERSATIONAL AI FRAMEWORK FOR CONTINUOUS INDIAN SIGN LANGUAGE COMMUNICATION

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Durga.G

Vels Institute of Science,Technology and Advanced Studies Pallavaram

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

SIGNVISION: A REAL-TIME CIMPUTER AND CONVERSATIONAL AI FRAMEWORK FOR CONTINUOUS INDIAN SIGN LANGUAGE COMMUNICATION

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Deepak.A

Vels Institute of Science,Technology and Advanced Studies Pallavaram

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

SIGNVISION: A REAL-TIME CIMPUTER AND CONVERSATIONAL AI FRAMEWORK FOR CONTINUOUS INDIAN SIGN LANGUAGE COMMUNICATION

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Mukesh.P

Vels Institute of Science,Technology and Advanced Studies Pallavaram

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

SIGNVISION: A REAL-TIME CIMPUTER AND CONVERSATIONAL AI FRAMEWORK FOR CONTINUOUS INDIAN SIGN LANGUAGE COMMUNICATION

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh